

Análise de credibilidade dos dados de fornecedores

Aqui eu olho a qualidade do cadastro em quatro pontos:

1. **Compleitude** — se todo fornecedor tem CNPJ ou CPF
2. **Unicidade** — CNPJ repetido e gente com CNPJ e CPF no mesmo registro
3. **Consistência** — se a UF existe na base de municípios
4. **Similaridade** — se a descrição do CNAE bate com a tabela da Receita (e o quão diferente ela é, via Levenshtein)

1. Imports

```
In [47]: import os
import html as html_lib
import pandas as pd
from IPython.display import HTML, display

try:
    import Levenshtein

    def calcular levenshtein(s1, s2):
        return Levenshtein.distance(str(s1), str(s2))
except ImportError:
    def calcular levenshtein(s1, s2):
        s1, s2 = str(s1), str(s2)
        m, n = len(s1), len(s2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(m + 1):
            dp[i][0] = i
        for j in range(n + 1):
            dp[0][j] = j
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if s1[i - 1] == s2[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1]
                else:
                    dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])

        return dp[m][n]

pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", None)

# --- saidas em html (uso isso no resto do notebook) ---
_CSS = """
.saida{font-family:"Segoe UI",system-ui,sans-serif;color:#e7e2d6;background:#141a22;
```

```

border:1px solid #2b3340;border-radius:16px;padding:18px 20px;margin:10px 0 18px}
.saida *{box-sizing:border-box}
.saida h3{margin:0 0 14px;font-size:12px;letter-spacing:.16em;text-
transform:uppercase;
color:#d4b07a;font-weight:700}
.saida-kpis{display:grid;grid-template-columns:repeat(auto-
fit,minmax(148px,1fr));gap:10px}
.saida-kpi{background:#1c2430;border-radius:12px;padding:12px 14px;border-
left:3px solid #d4b07a}
.saida-kpi.ok{border-left-color:#6cae8c}
.saida-kpi.warn{border-left-color:#d46b4c}
.saida-kpi.info{border-left-color:#6e8fbf}
.saida-kpi .v{font-family:Consolas,"Cascadia Mono",monospace;font-size:22px;font-
weight:700;line-height:1.15}
.saida-kpi .l{font-size:11px;letter-spacing:.07em;text-
transform:uppercase;color:#8d95a3;margin-top:5px}
.saida-kpi .s{font-size:12px;color:#b7b3a8;margin-top:4px}
.saida-note{margin:0;color:#b7b3a8;font-size:13.5px;line-height:1.45}
.saida-chips{display:flex;flex-wrap:wrap;gap:6px}
.saida-chip{background:#1c2430;border:1px solid #2b3340;border-
radius:999px;padding:4px 10px;
font-family:Consolas,"Cascadia Mono",monospace;font-size:11px;color:#e7e2d6}
.saida-bar{height:7px;background:#2b3340;border-
radius:99px;overflow:hidden;margin-top:10px}
.saida-bar>i{display:block;height:100%;background:#d4b07a}
.saida-bar.warn>i{background:#d46b4c}
.saida-bar.ok>i{background:#6cae8c}
.saida-scroll{max-height:440px;overflow:auto;border-radius:10px;border:1px solid
#2b3340}
.saida table{width:100%;border-collapse:collapse;font-size:12.5px}
.saida th{text-align:left;color:#d4b07a;font-size:10.5px;letter-
spacing:.08em;text-transform:uppercase;
padding:9px 10px;border-bottom:1px solid
#2b3340;position:sticky;top:0;background:#1a212c}
.saida td{padding:8px 10px;border-bottom:1px solid #222a35;vertical-
align:top;color:#e7e2d6}
.saida tr:nth-child(even) td{background:#181f29}
.saida tr:hover td{background:#222b38}
.saida .muted{color:#8d95a3;font-size:12px;margin:8px 0 0}
"""

```

```

def _esc(v):
    if v is None or (isinstance(v, float) and pd.isna(v)):
        return "_"
    return html_lib.escape(str(v))

```

```

def _pinta(html_body):
    display(HTML(f"<style>{_CSS}</style>{html_body}"))

```

```

def show_kpis(titulo, itens):
    cards = []
    for item in itens:
        tom = item.get("tom", "")

```

```

        sub = f"<div class='s'>{_esc(item['sub'])}</div>" if item.get("sub") else
"""
        cards.append(
            f"<div class='saida-kpi {tom}'><div class='v'>{_esc(item['valor'])}</div>"
        </div>"
            f"<div class='l'>{_esc(item['label'])}</div>{sub}</div>"
        )
        _pinta(f"<div class='saida'><h3>{_esc(titulo)}</h3><div class='saida-kpis'>
{''.join(cards)}</div></div>")

def show_msg(titulo, texto, tom="info"):
    _pinta(
        f"<div class='saida'><h3>{_esc(titulo)}</h3>"
        f"<p class='saida-note'>{_esc(texto)}</p></div>"
    )

def show_chips(titulo, valores):
    chips = "".join(f"<span class='saida-chip'>{_esc(v)}</span>" for v in
valores)
    _pinta(f"<div class='saida'><h3>{_esc(titulo)}</h3><div class='saida-chips'>
{chips}</div></div>")

def show_barra(titulo, pct, texto, tom=""):
    pct = max(0, min(100, float(pct)))
    _pinta(
        f"<div class='saida'><h3>{_esc(titulo)}</h3>"
        f"<p class='saida-note'>{_esc(texto)}</p>"
        f"<div class='saida-bar {tom}'><i style='width:{pct:.2f}%></i></div>"
        f"<p class='muted'>{pct:.2f}%</p></div>"
    )

def show_tabela(df, titulo, n=10):
    if df is None or len(df) == 0:
        show_msg(titulo, "nenhuma linha pra mostrar")
        return
    view = df.head(n)
    th = "".join(f"<th>{_esc(c)}</th>" for c in view.columns)
    rows = []
    for _, row in view.iterrows():
        tds = "".join(f"<td>{_esc(v)}</td>" for v in row.tolist())
        rows.append(f"<tr>{tds}</tr>")
    extra = f"<p class='muted'>mostrando {len(view)} de {len(df)}</p>" if len(df)
> n else ""
    _pinta(
        f"<div class='saida'><h3>{_esc(titulo)}</h3>"
        f"<div class='saida-scroll'><table><thead><tr>{th}</tr></thead>"
        f"<tbody>{''.join(rows)}</tbody></table></div>{extra}</div>"
    )

```

2. Lendo as bases

Os arquivos estão em `data/Dados` : fornecedores, municípios, CNAE interno e o CSV da RFB.

```
In [48]: def encontrar_arquivo(nome_arquivo):
    candidatos = [
        nome_arquivo,
        os.path.join("data", nome_arquivo),
        os.path.join("data", "Dados", nome_arquivo),
        os.path.join("data", "TrabalhoFinal", nome_arquivo),
        os.path.join("../", "data", nome_arquivo),
    ]
    for caminho in candidatos:
        if os.path.exists(caminho):
            return caminho
    raise FileNotFoundError(
        f"Não achei '{nome_arquivo}'. Olhei em:\n- " + "\n- ".join(candidatos)
    )

df_fornecedores = pd.read_pickle(encontrar_arquivo("fornecedores_transf.pkl"))
df_municipios = pd.read_pickle(encontrar_arquivo("municipios_transf.pkl"))
df_cnaes_transf = pd.read_pickle(encontrar_arquivo("cnaes_transf.pkl"))

caminho_cnae_rfb = encontrar_arquivo("CNAERFB.CSV")
try:
    df_cnaes_rfb = pd.read_csv(
        caminho_cnae_rfb,
        sep=";",
        encoding="latin1",
        header=None,
        names=["codigo", "descricao"],
        dtype=str,
    )
except (UnicodeDecodeError, pd.errors.ParserError):
    df_cnaes_rfb = pd.read_csv(
        caminho_cnae_rfb,
        sep=",",
        encoding="utf-8",
        header=None,
        names=["codigo", "descricao"],
        dtype=str,
    )

show_kpis(
    "bases carregadas",
    [
        {"label": "fornecedores", "valor": f"
{df_fornecedores.shape[0]:,}".replace(",", "."), "sub": f"
{df_fornecedores.shape[1]} colunas", "tom": "info"},
        {"label": "municipios", "valor": f"
{df_municipios.shape[0]:,}".replace(",", "."), "sub": f"{df_municipios.shape[1]}
colunas", "tom": "info"},
        {"label": "cnae interno", "valor": f"
{df_cnaes_transf.shape[0]:,}".replace(",", "."), "sub": f"
{df_cnaes_transf.shape[1]} colunas", "tom": "info"},
        {"label": "cnae rfb", "valor": f"{df_cnaes_rfb.shape[0]:,}".replace(",",
```

```

        "."), "sub": f"{df_cnaes_rfb.shape[1]} colunas", "tom": "info"},
    ],
)
show_chips("colunas dos fornecedores", df_fornecedores.columns.tolist())
esquema = pd.DataFrame(
    {
        "coluna": df_fornecedores.columns,
        "tipo": df_fornecedores.dtypes.astype(str).values,
        "preenchidos": df_fornecedores.notna().sum().values,
    }
)
show_tabela(esquema, "esquema de fornecedores", n=ler(esquema))

```

BASES CARREGADAS

15.734

FORNECEDORES

15 colunas

5.584

MUNICÍPIOS

6 colunas

1.348

CNAE INTERNO

3 colunas

1.359

CNAE RFB

2 colunas

COLUNAS DOS FORNECEDORES

id

cnnpj

cpf

nome

ativo

recadastrado

id_municipio

uf

id_natureza_juridica

id_porte_empresa

id_amo_negocio

id_unidade_cadastradora

id_cnae

id_cnae2

habilitado_licitar

ESQUEMA DE FORNECEDORES

COLUNA	TIPO	PREENCHIDOS
id	int64	15734
cnpj	object	13410
cpf	object	2324
nome	object	15734
ativo	bool	15734
recadastrado	object	0
id_municipio	int64	15734
uf	object	15734
id_natureza_juridica	float64	13393
id_porte_empresa	float64	15710
id_ramo_negocio	object	0
id_unidade_cadastradora	object	0

```
In [40]: show_tabela(df_fornecedores, "amostra de fornecedores", n=5)
```

AMOSTRA DE FORNECEDORES

ID	CNPJ	CPF	NOM	ATIV	RECA	ID_M	UF	ID_N	ID_P	ID_R	ID_U	ID_C	ID_C	HAB
498	—	***635889710e	CONSTRUTORA SILVEIRA	—	—	35	RO	—	1.0	—	—	—	—	True
2597	00265426000107	000000000000000000	CONCRETA ENGENHARIA E CONSTRUCOES LTDA	—	—	35	RO	2.0	2.0	—	—	4120400-0	4120400-0	True
3519	00351475000102	000000000000000000	TOPELEB COM DE PECAS E LUBRIFICANTES LTDA	False	—	94	RO	2.0	2.0	—	—	4732600-0	4732600-0	True
4603	00449404000147	000000000000000000	AGUMMA COMERCIO E SERVICOS LTDA	False	—	35	RO	2.0	2.0	—	—	4751204-0	4751204-0	True
4605	00449484000100	000000000000000000	COMERCIO	—	—	132	RO	2.0	2.0	—	—	4681802-0	4681802-0	True

mostrando 5 de 15734

3. Completude

Todo fornecedor precisa ter CNPJ ou CPF. Se os dois campos vierem vazios, o registro não passa.

```
In [41]: # pego as colunas mesmo se o nome vier em minúsculo
col_cnpj = next((c for c in df_fornecedores.columns if "CNPJ" in c.upper()),
"CNPJ")
col_cpf = next((c for c in df_fornecedores.columns if "CPF" in c.upper()), "CPF")

# astype(str) transforma NaN em "nan", então trato isso como vazio também
cnpj_vazio = df_fornecedores[col_cnpj].isna() | (
    df_fornecedores[col_cnpj].astype(str).str.strip().isin(["", "nan", "None"])
)
cpf_vazio = df_fornecedores[col_cpf].isna() | (
    df_fornecedores[col_cpf].astype(str).str.strip().isin(["", "nan", "None"])
)

df_sem_documento = df_fornecedores[~(cnpj_vazio & cpf_vazio)]
```

```

total_fornecedores = len(df_fornecedores)
total_sem_doc = len(df_sem_documento)
taxa_completude = ((total_fornecedores - total_sem_doc) / total_fornecedores) *
100

show_kpis(
    "completude",
    [
        {"label": "fornecedores", "valor": f"{total_fornecedores:,}".replace(",",
"."), "tom": "info"},
        {"label": "sem documento", "valor": total_sem_doc, "tom": "warn" if
total_sem_doc else "ok"},
        {"label": "completude", "valor": f"{taxa_completude:.2f}%", "tom": "ok"
if taxa_completude == 100 else "warn"},
    ],
)
show_barra("taxa de completude", taxa_completude, "quem tem CNPJ ou CPF
preenchido", "ok" if taxa_completude == 100 else "warn")
if total_sem_doc > 0:
    show_tabela(df_sem_documento, "registros sem documento", n=10)
else:
    show_msg("completude", "ninguém ficou sem documento")

```

COMPLETUDE

15.734

FORNECEDORES

0

SEM DOCUMENTO

100.00%

COMPLETUDE

TAXA DE COMPLETUDE

quem tem CNPJ ou CPF preenchido



100.00%

COMPLETUDE

ninguém ficou sem documento

4. Unicidade

Duas checagens: se o mesmo CNPJ aparece em mais de um fornecedor, e se algum registro tem CNPJ e CPF juntos (PF e PJ no mesmo cadastro).


```
In [42]: # tiro pontuação do CNPJ pra não contar o mesmo número duas vezes
cnpjjs_preenchidos = df_fornecedores[~cnpj_vazio].copy()
cnpjjs_preenchidos["_cnpj_norm"] = (
    cnpjjs_preenchidos[col_cnpj].astype(str).str.replace(r"\D", "", regex=True)
)
cnpjjs_duplicados = cnpjjs_preenchidos[
    cnpjjs_preenchidos.duplicated(subset=["_cnpj_norm"], keep=False)
]

df_cnpj_e_cpf = df_fornecedores[(~cnpj_vazio) & (~cpf_vazio)]

show_kpis(
    "unicidade",
    [
        {"label": "cnpj duplicado", "valor": len(cnpjjs_duplicados), "tom": "warn"
         if len(cnpjjs_duplicados) else "ok"},
        {"label": "cnpj + cpf juntos", "valor": len(df_cnpj_e_cpf), "tom": "warn"
         if len(df_cnpj_e_cpf) else "ok"},
    ],
)
if len(cnpjjs_duplicados) > 0:
    show_tabela(cnpjjs_duplicados.sort_values(by=col_cnpj), "cnpjjs repetidos",
n=10)
else:
    show_msg("cnpj duplicado", "não achei o mesmo CNPJ em mais de um fornecedor")
if len(df_cnpj_e_cpf) > 0:
    show_tabela(df_cnpj_e_cpf, "registros com cnpj e cpf", n=10)
else:
    show_msg("pf / pj", "ninguém misturou CNPJ e CPF no mesmo cadastro")
```

UNICIDADE

0

CNPJ DUPLICADO

0

CNPJ + CPF JUNTOS

CNPJ DUPLICADO

não achei o mesmo CNPJ em mais de um fornecedor

PF / PJ

ninguém misturou CNPJ e CPF no mesmo cadastro

5. Consistência de UF

A UF do fornecedor tem que existir na base de municípios. Qualquer sigla fora dessa lista (ou nula) eu marco como inconsistente.

```
In [43]: # na base de municípios a coluna veio como sigla_uf
col_uf_forn = next(
    (: for c in df_fornecedores.columns if c.upper() in ["UF", "SIGLA_UF",
"ESTADO", "UF_FORNECEDOR"]),
    "UF",
)
col_uf_mun = next(
    (: for c in df_municipios.columns if c.upper() in ["UF", "SIGLA_UF",
"ESTADO", "SG_UF"]),
    "UF",
)

ufs_validas =
set(df_municipios[col_uf_mun].dropna().astype(str).str.strip().str.upper().unique())
ufs_forn = df_fornecedores[col_uf_forn].astype(str).str.strip().str.upper()
df_uf_inconsistente = df_fornecedores[~ufs_forn.isin(ufs_validas)]
taxa_conformidade_uf = ((total_fornecedores - len(df_uf_inconsistente)) /
total_fornecedores) * 100

show_kpis(
    "consistência de uf",
    [
        {"label": "ufs de referência", "valor": len(ufs_validas), "tom": "info"},
        {"label": "uf estranha", "valor": len(df_uf_inconsistente), "tom": "warn"}
    ],
    if len(df_uf_inconsistente) else "ok",
    {"label": "conformidade", "valor": f"{taxa_conformidade_uf:.2f}%", "tom":
"ok" if taxa_conformidade_uf == 100 else "warn"},
    ],
)
show_chips("ufs da base de municípios", sorted(ufs_validas))
show_barra("conformidade da uf", taxa_conformidade_uf, "fornecedores cuja UF
existe na tabela de municípios", "ok" if taxa_conformidade_uf == 100 else "warn")
if len(df_uf_inconsistente) > 0:
    show_chips("valores que não bateram",
df_fornecedores.loc[~ufs_forn.isin(ufs_validas), col_uf_forn].unique())
    show_tabela(df_uf_inconsistente, "fornecedores com uf inconsistente", n=10)
```

CONSISTÊNCIA DE UF

27

UFS DE REFERÊNCIA

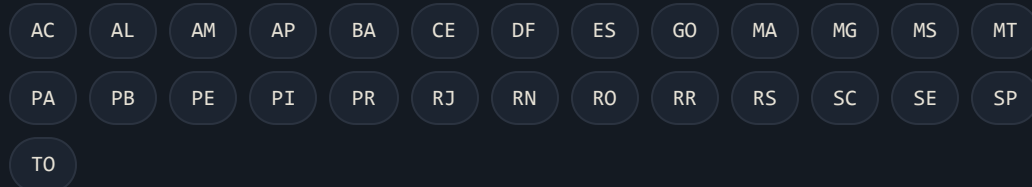
0

UF ESTRANHA

100.00%

CONFORMIDADE

UFS DA BASE DE MUNICÍPIOS



CONFORMIDADE DA UF

fornecedores cuja UF existe na tabela de municípios



100.00%

6. CNAE — bate com a Receita?

Cruzo o CNAE interno com o CSV da RFB pelo código e vejo se a descrição é a mesma. O CSV não tem cabeçalho, então a primeira linha já é dado.

```
In [44]: def padronizar_cnae(serie):
# 0111-3/01 vira 0111301
return serie.astype(str).str.replace(r"\D", "",
regex=True).str.strip().str.zfill(7)

col_cod_transf = "codigo_longo" if "codigo_longo" in df_cnaes_transf.columns else
df_cnaes_transf.columns[0]
col_desc_transf = "descricao" if "descricao" in df_cnaes_transf.columns else
df_cnaes_transf.columns[1]
col_cod_rfb = "codigo" if "codigo" in df_cnaes_rfb.columns else
df_cnaes_rfb.columns[0]
col_desc_rfb = "descricao" if "descricao" in df_cnaes_rfb.columns else
df_cnaes_rfb.columns[1]

df_cnaes_transf["COD_CNAE_LIMPO"] =
padronizar_cnae(df_cnaes_transf[col_cod_transf])
df_cnaes_rfb["COD_CNAE_LIMPO"] = padronizar_cnae(df_cnaes_rfb[col_cod_rfb])

# left pra eu ver também quem não existe na RFB
df_cnaes_merge = pd.merge(
    df_cnaes_transf[["COD_CNAE_LIMPO", col_desc_transf]].rename(
        columns={col_desc_transf: "Descricao_Fornecedor"}
    ),
    df_cnaes_rfb[["COD_CNAE_LIMPO", col_desc_rfb]].rename(
        columns={col_desc_rfb: "Descricao_RFB"}
    ),
    on="COD_CNAE_LIMPO",
```

```

        hov="left",
    )

df_cnaes_sem_rfb = df_cnaes_merge[df_cnaes_merge["Descricao_RFB"].isna()].copy()
df_cnaes_comparados =
df_cnaes_merge[df_cnaes_merge["Descricao_RFB"].notna()].copy()

df_cnaes_divergentes = df_cnaes_comparados[

df_cnaes_comparados["Descricao_Fornecedor"].astype(str).str.strip().str.upper()
    != df_cnaes_comparados["Descricao_RFB"].astype(str).str.strip().str.upper()
].copy()

pct_div = (len(df_cnaes_divergentes) / len(df_cnaes_comparados) * 100) if
len(df_cnaes_comparados) else 0
show_kpis(
    "cnae vs receita",
    [
        {"label": "interno", "valor": len(df_cnaes_transf), "tom": "info"},
        {"label": "tabela rfb", "valor": len(df_cnaes_rfb), "tom": "info"},
        {"label": "deu match", "valor": len(df_cnaes_comparados), "tom": "ok"},
        {"label": "sem par na rfb", "valor": len(df_cnaes_sem_rfb), "tom": "warn"}
    if len(df_cnaes_sem_rfb) else "ok"},
        {"label": "texto diferente", "valor": len(df_cnaes_divergentes), "tom":
"warn" if len(df_cnaes_divergentes) else "ok"},
    ],
)
show_barra("quanto do cnae interno diverge", pct_div, "mesmo código, descrição
diferente da RFB", "warn" if pct_div else "ok")
show_tabela(
    df_cnaes_divergentes.rename(columns={"COD_CNAE_LIMPO": "Codigo_CNAE"}),
    "amostra das descrições diferentes",
    n=10,
)
if len(df_cnaes_sem_rfb) > 0:
    show_tabela(
        df_cnaes_sem_rfb.rename(columns={"COD_CNAE_LIMPO": "Codigo_CNAE"}),
        "códigos internos sem par na rfb",
        n=10,
    )

```

CNAE VS RECEITA

1348

INTERNO

1359

TABELA RFB

1348

DEU MATCH

0

SEM PAR NA RFB

1207

TEXTO DIFERENTE

QUANTO DO CNAE INTERNO DIVERGE

mesmo código, descrição diferente da RFB

89.54%

AMOSTRA DAS DESCRIÇÕES DIFERENTES

CODIGO_CNAE	DESCRICAO_FORNECEDOR	DESCRICAO_RFB
0111399	CULTIVO DE OUTROS CEREAIS NAO ESPECIFICADOS ANTERIORMENTE	Cultivo de outros cereais não especificados anteriormente
0112101	CULTIVO DE ALGODAO HERBACEO	Cultivo de algodão herbáceo
0112199	CULTIVO DE OUTRAS FIBRAS DE LAVOURA TEMPORARIA NAO ESPECIFICADAS ANTERIORMENTE	Cultivo de outras fibras de lavoura temporária não especificadas anteriormente
0113000	CULTIVO DE CANA-DE-ACUCAR	Cultivo de cana-de-açúcar
0116499	CULTIVO DE OUTRAS OLEAGINOSAS DE LAVOURA TEMPORARIA NAO ESPECIFICADAS ANTERIORMENTE	Cultivo de outras oleaginosas de lavoura temporária não especificadas anteriormente
0119905	CULTIVO DE FEIJAO	Cultivo de feijão

mostrando 10 de 1207

7. Distância de Levenshtein

Nos CNAEs que não bateram, calculo quantas edições separam as duas descrições e transformo isso numa similaridade de 0 a 100%. Comparo em maiúsculo pra não inflar a distância só por causa de caixa.

```
In [45]: if len(df_cnaes_divergentes) > 0:
          # upper pra distância não crescer só porque um lado está em CAPS
          desc_f =
df_cnaes_divergentes["Descricao_Fornecedor"].astype(str).str.strip().str.upper()
          desc_r =
```

```

df_cnaes_divergentes["Descricao_RFB"].astype(str).str.strip().str.upper()

df_cnaes_divergentes["Distancia_Levenshtein"] = [
    calcular levenshtein(a, b) for a, b in zip(desc_f, desc_r)
]
tamanhos = [max(len(a), len(b)) for a, b in zip(desc_f, desc_r)]
df_cnaes_divergentes["Grau_Similaridade_%"] = [
    round((1 - dist / tam) * 100, 2) if tam else 100.0
    for dist, tam in zip(df_cnaes_divergentes["Distancia_Levenshtein"],
tamanhos)
]

df_resultado levenshtein = (
    df_cnaes_divergentes.rename(columns={"COD_CNAE_LIMPO": "Codigo_CNAE"})[
        [
            "Codigo_CNAE",
            "Descricao_Fornecedor",
            "Descricao_RFB",
            "Distancia_Levenshtein",
            "Grau_Similaridade_%",
        ]
    ]
    .sort_values("Distancia_Levenshtein", ascending=False)
    .reset_index(drop=True)
)

show_kpis(
    "levenshtein",
    [
        {"label": "códigos", "valor": len(df_resultado levenshtein), "tom":
"warn"},
        {"label": "similaridade média", "valor": f"
{df_resultado levenshtein['Grau_Similaridade_%'].mean():.2f}%", "tom": "info"},
        {"label": "mínima", "valor": f"
{df_resultado levenshtein['Grau_Similaridade_%'].min():.2f}%", "tom": "warn"},
        {"label": "maior distância", "valor":
int(df_resultado levenshtein["Distancia_Levenshtein"].max()), "tom": "warn"},
    ],
)
show_barra(
    "similaridade média",
    df_resultado levenshtein["Grau_Similaridade_%"].mean(),
    "média do grau de similaridade nas descrições que divergiram",
    "ok",
)
show_tabela(df_resultado levenshtein, "piores divergências (maior
distância)", n=12)
else:
    df_resultado levenshtein = pd.DataFrame(
        columns=[
            "Codigo_CNAE",
            "Descricao_Fornecedor",
            "Descricao_RFB",
            "Distancia_Levenshtein",
            "Grau_Similaridade_%",
        ]
    )

```

```
)  
show_msg("levenshtein", "as descrições são iguais, não tem o que calcular")
```

LEVENSHTEIN

1207

CÓDIGOS

93.86%

SIMILARIDADE MÉDIA

60.34%

MÍNIMA

33

MAIOR DISTÂNCIA

SIMILARIDADE MÉDIA

média do grau de similaridade nas descrições que divergiram



93.86%

PIORES DIVERGÊNCIAS (MAIOR DISTÂNCIA)

CODIGO_CNAE	DESCRICAO_FORN	DESCRICAO_RFB	DISTANCIA_LEVEL	GRAU_SIMILARID
8720499	ATIVIDADES DE ASSISTENCIA PSICOSSOCIAL E A SAUDE A PORTADORES DE DISTURBIOS PSQUICOS, DEFICIENCIA MENTAL E DEPENDENCIA QUIMICA NAO ESPECIFICADAS ANTERIORMENTE	Atividades de assistência psicossocial e à saúde a portadores de distúrbios psíquicos, deficiência mental e dependência química e grupos similares não	33	79.25
1741902	FABRICACAO DE PRODUTOS DE PAPEL, CARTOLINA, PAPEL-CARTAO E PAPELAO ONDULADO PARA USO COMERCIAL E DE ESCRITORIO	Fabricação de produtos de papel, cartolina, papel cartão e papelão ondulado para uso comercial e de escritório, exceto formulário	33	76.09

mostrando 12 de 1207

8. O que eu achei

No cadastro em si a base está ok: os 15.734 fornecedores têm documento, não achei CNPJ duplicado nem PF/PJ misturado, e as UFs fecham 100% com a tabela de municípios.

O buraco é o CNAE. Dos 1.348 códigos internos, 1.207 têm texto diferente da Receita. Na prática é acento e grafia (`ALGODAO` vs `algodão`); a similaridade média ficou em 93,86%. Mesmo assim, a descrição oficial deveria ser a referência.

A célula de baixo só junta os números pra não ficar caçando print nas seções.

```
In [46]: show_kpis(  
    "resumo",  
    [  
        {"label": "fornecedores", "valor": f"{total_fornecedores:,}".replace(",",  
        ".")", "tom": "info"},  
        {"label": "completude", "valor": f"{taxa_completude:.2f}%", "sub": f"  
{total_sem_doc} sem documento", "tom": "ok" if total_sem_doc == 0 else "warn"},  
        {"label": "cnpj duplicado", "valor": len(cnpjs_duplicados), "tom": "ok"
```



```

if len(cnpjs_duplicados) == 0 else "warn"},
    {"label": "cnpj + cpf", "valor": len(df_cnpj_e_cpf), "tom": "ok" if
len(df_cnpj_e_cpf) == 0 else "warn"},
    {"label": "uf ok", "valor": f"{taxa_conformidade_uf:.2f}%", "tom": "ok"
if taxa_conformidade_uf == 100 else "warn"},
    {
        "label": "cnae diferente",
        "valor": len(df_cnaes_divergentes),
        "sub": f"{len(df_cnaes_comparados)} comparados",
        "tom": "warn" if len(df_cnaes_divergentes) else "ok",
    },
],
)
)
if len(df_resultado levenshtein) > 0:
    show_barra(
        "similaridade média do cnae",
        df_resultado levenshtein["Grau_Similaridade_%"].mean(),
        "as descrições quase só mudam acento e grafia, mas ainda assim não são
iguais",
        "ok",
    )
    show_tabela(df_resultado levenshtein, "top 5 mais distantes", n=5)

```

RESUMO

15.734

FORNECEDORES

100.00%

COMPLETUDE

0 sem documento

0

CNPJ DUPLICADO

0

CNPJ + CPF

100.00%

UF OK

1207

CNAE DIFERENTE

1348 comparados

SIMILARIDADE MÉDIA DO CNAE

as descrições quase só mudam acento e grafia, mas ainda assim não são iguais



93.86%

TOP 5 MAIS DISTANTES

CODIGO_CNAE	DESCRICAO_FORN	DESCRICAO_RFB	DISTANCIA_LEVEL	GRAU_SIMILARID
8720499	ATIVIDADES DE ASSISTENCIA PSICOSSOCIAL E A SAUDE A PORTADORES DE DISTURBIOS PSQUICOS, DEFICIENCIA MENTAL E DEPENDENCIA QUIMICA NAO ESPECIFICADAS ANTERIORMENTE	Atividades de assistência psicossocial e à saúde a portadores de distúrbios psíquicos, deficiência mental e dependência química e grupos similares não	33	79.25
1741902	FABRICACAO DE PRODUTOS DE PAPEL, CARTOLINA, PAPEL-CARTAO E PAPELAO ONDULADO PARA USO COMERCIAL E DE ESCRITORIO	Fabricação de produtos de papel, cartolina, papel cartão e papelão ondulado para uso comercial e de escritório, exceto formulário	33	76.09

mostrando 5 de 1207